1 Architecture

The operator reaches the static dashboard and console API on the Go HTTP server. API handlers use the adapter's own identity for HTTPS provider calls. Runtime inspection/control uses a Unix-socket Docker client. Optional telemetry is published directly to AMQP. The package contains the original shared Go runtime because TLS/client/model/registration helpers are interdependent; the dedicated executable rejects other role selections.

2 Components

`cmd/localcloudmanager/main.go`: role-constrained launcher. `internal/cloud/localcloudmanager.go`: sessions, API aggregation, provider adapters, Docker operations. `assets.go` and `ui/`: compile-time embedded UI, retaining explicit UI_ROOT and legacy on-disk fallbacks. `app.go`: environment, server, registration and provider-call retry. `security.go`: certificates, CRL and identity. `monitoring.go`: AMQPS publisher. `model.go`: inherited JSON state. Other shared files remain compile-time dependencies, not newly deployed support systems.

3 Request sequence and trust boundaries

Transport mTLS is enforced when secure mode is selected. Static assets and health need no operator session. For `/api/`, Host/Origin policy and configured console password are checked, session requests are handled, then a valid operator cookie is required. Mutations additionally require `X-Arrowhead-Action:confirm`. These handlers run before the shared Arrowhead token/management dispatch; a bearer token does not replace a browser session. Provider calls obtain the manager's own token and refresh once after401. Network effects and local state are not a distributed transaction.

4 State and recovery

Console sessions and the last60 overview samples live in process memory and are lost on restart. Shared JSON state uses a temporary sibling file and rename, but no new fsync/process-lock guarantees are introduced. Registration retries every20seconds. Displayed registered/naming flags are synthetic and not evidence of successful registration. Provider/Docker failures appear in collected responses or502. Do not share active cloud state files with this package.

5 Security and configuration

Authentication server CN/profile are explicitly pinned before credentials; other provider peers have CA/SAN validation without universal expected-CN pinning. Optional CRLs are checked for new connections. Console cookies are HttpOnly/SameSite=Strict but not Secure or port-isolated; Origin checks do not compare exact ports. Lifecycle uses the static systemToCompose allowlist (MANAGED_SERVICES is not consumed); log reads are project-scoped, not restricted to that allowlist. A raw Docker socket is a privileged trust boundary; deployment isolation or a least-privilege proxy remains necessary. See README.md for every environment variable and COMPATIBILITY.md for release blockers.

6 Compatibility and release gates

This relocation intentionally preserves the latest located manager's business behavior. New changes are packaging (standalone nested module/role-specific launcher), embedded UI fallback, documentation and tests. This is not a security remediation or full Java-equivalence implementation.

1. `PUT/DELETE/api/sysmonitor/subscriptions/{providerSystem}` currently checks confirmation and returns updated/removed flags without forwarding a write. Do not interpret this as applied configuration. This remains a release blocker.

2. `/api/intercloud` returns a static unavailable/no-relay summary. It does not inspect Gatekeeper/GateTunnel. Consumer cloud labels include hard-coded TestCloud strings; no live foreign authorization catalogue is queried.

3. Overview hard-codes registration/naming flags and leaves several collections empty. Subscription status hard-codes intake.connected=true/providerFound=false. These are display adapters, not authoritative readiness or security evidence.

4. Historian, SysMonitor, plant and orchestration adapters target the inherited Go-cloud routes/shapes. They are not proven compatible with the newer generic Go support port or latest Java APIs. The standalone module must be qualified against the providers actually deployed.

5. Console Host/Origin checks compare host names, not exact origins/ports. The `ah_console` cookie is not port-isolated and lacks Secure. Sessions/history are in-memory and reset on restart. Missing password disables APIs. Login lockout is global, five failures in five minutes then one-minute lockout; this is not comprehensive abuse protection.

6. The manager API is dispatched before the shared application's token/management checks. Its boundary is console password/session plus the transport's mTLS if enabled, not a bearer-token-only API. Static assets and health have no console session requirement. A proxy must preserve this intended boundary.

7. Authentication has explicit CN/profile pinning. Other provider clients use CA/SAN validation but do not generally pin expected provider CN. Do not claim universal authority-substitution resistance for this inherited code. Redirects are disabled. Optional CRLs reload for new connections only; omitted CRLs mean no revocation enforcement.

8. Lifecycle uses the static systemToCompose service allowlist and Compose-project filtered container list; MANAGED_SERVICES is not consumed by this implementation. Action is start/stop/restart only. Logs, however, can read any returned project container, not just the lifecycle allowlist. The raw Docker socket still confers broad daemon privilege: application checks are not a least-privilege socket boundary. Production needs a constrained proxy and deployment review.

9. Shared runtime source includes other roles for compile-time dependencies and inherited tests. The dedicated executable refuses selecting those roles; they are not a second support-system deployment. No Docker socket or live state is needed by tests.

10. Telemetry AMQP publisher has no durable local outbox; retries/reconnects do not establish exactly-once delivery. power=line/available and process health are generated status, not measurements of physical hardware. Registration is retried every20seconds; displayed registration flags are not confirmation. No real broker test is claimed here.

11. Shared state is the inherited JSON format, not the newer support port's transactional state model. There is no new Java SQL migration, encryption-at-rest or multi-process ownership guarantee. Keep independent state and backups.

12. Historical Markdown/SysML claims of three-way interchangeability are retained only under docs/legacy. The revised model and new SysD/SysDD/SD/IDD set distinguish intended common contracts from actual Go qualification.

13. The shared serviceOperations helper generates POST/name metadata for localCloudManagement and the legacy monitor service, whereas actual handlers include GET/PUT/PATCH/DELETE and nested paths. Registration metadata is therefore not an authoritative HTTP contract. The new IDDs document actual handler methods/paths; correcting advertised metadata is separate functional work, not silently folded into relocation.

Source relocation does not modify running containers or remove their deployment source. Before switching deployment build paths, close these gaps, qualify provider DTOs/policies, review Docker access, isolate browser trust domains, and test rollback.

7 References and verification

Authoritative moved code: LocalCloudManager/internal/cloud and embedded ui sources. Exact original/relocated SHA256 hashes are in docs/source-manifest.json. docs/legacy preserves the earlier local Markdown/SysML, not an endorsement of its parity claims. The revised SysML variability model is descriptive and has not been checked by a SysML parser. Executed checks and qualification limits are recorded in VERIFICATION.md. Source movement does not deploy or alter running clouds.